SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 11: Task C.7

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – April-2026

Contents

1. Overview 3
2. Implementation Summary 3
2.1 Data collection and preprocessing 3
2.2 Sentiment analysis methods 5
2.3 Key functions and modules added for Task C.7 6
3. Less Straightforward Design Decisions 7
3.1 Time alignment between news and stock data 7
3.2 Label construction for next-day direction 7
3.3 Why both VADER and FinBERT were used 8
3.4 Time-based train/test split and evaluation metrics 8
4. References Used 9
5 Experiment Results Summary 9
5.1 Dataset summary 9
5.2 Model comparison results 10
5.3 Interpretation of results 11
5.4 Challenges and limitations 12
6. Visualisations Included 12
7. Conclusion 15

1. Overview

The aim of Task C.7 was to extend the stock prediction work from earlier tasks by incorporating sentiment analysis from external textual sources and converting the problem into a binary classification task. Instead of forecasting an exact closing price, the objective here was to predict whether the stock price would move UP or DOWN on the next trading day.

For this task, the target stock was CBA.AX, with historical OHLCV price data collected from Yahoo Finance through yfinance. News text was collected from two external sources: FinViz and the Alpha Vantage News Sentiment API. Sentiment scores were generated from daily news headlines using both VADER and FinBERT, then merged with price-based features to train classification models. The final system compared price-only models against price + sentiment models to determine whether sentiment features added predictive value.

This task also included an independent research component by integrating FinBERT, which is a domain-specific transformer-based model for financial sentiment, and comparing its performance against the simpler VADER baseline.

2. Implementation Summary

2.1 Data collection and preprocessing

The implementation used two main data streams:

1. Historical stock prices for CBA.AX from Yahoo Finance
2. Financial news headlines from FinViz and Alpha Vantage

The stock data loader used a caching system to avoid repeated downloads and also handled the common yfinance issue where the end= date is exclusive by adding one day to the requested end date before downloading. The downloaded price data was reduced to the standard OHLCV columns used later for feature engineering.

```python
def download_or_load_stock_data(
    company: str,
    start_date: str,
    end_date: str,
    cache_dir: str = "data_cache",
    use_cache: bool = True,
    force_download: bool = False,
) -> pd.DataFrame:
    """
    Download OHLCV stock data from yfinance OR load from local CSV cache.

    Why caching matters:
    - yfinance depends on network and can be slow.
    - caching saves time when rerunning experiments.

    If use_cache=True:
      - If the CSV exists and force_download=False, load it.
      - Otherwise download and save it.
    """
    os.makedirs(cache_dir, exist_ok=True)

    cache_name = _safe_filename(f"{company}_{start_date}_{end_date}.csv")
    cache_path = os.path.join(cache_dir, cache_name)

    if use_cache and (not force_download) and os.path.exists(cache_path):
        df = pd.read_csv(cache_path, index_col=0, parse_dates=True)
        print(f"Loaded from cache: {cache_path}")
    else:
        end_plus_1 = (pd.to_datetime(end_date) + pd.Timedelta(days=1)).strftime("%Y-%m-%d")
        df = yf.download(company, start=start_date, end=end_plus_1, progress=False)
        df = _flatten_yfinance_columns(df)

        if use_cache:
            df.to_csv(cache_path)
            print(f"Saved to cache: {cache_path}")

    # Ensure DatetimeIndex
    if not isinstance(df.index, pd.DatetimeIndex):
        df.index = pd.to_datetime(df.index)

    # Keep only standard OHLCV columns if present
    keep = [c for c in ["Open", "High", "Low", "Close", "Adj Close", "Volume"] if c in df.columns]
    df = df[keep].copy()

    return df
```

Price data download and caching logic for CBA.AX, including end-date adjustment for yfinance.

For the news component, FinViz headlines were scraped from the ticker page and stored locally as CSV. Alpha Vantage news was fetched in multiple date windows and cached as JSON files to reduce API load and avoid rate-limit issues. After collection, both sources were standardized into the same structure with columns such as date, headline, source, and url, then combined and deduplicated. In addition, Alpha Vantage news was filtered using a keyword list related to Australia, the ASX, the Reserve Bank of Australia, and banking terms.

```python
def fetch_finviz_headlines(ticker: str, out_csv: str, timeout: int = 20, use_cache: bool = True):
    """
    Scrape FinViz news table from quote page and save to CSV.
    Output columns: date, time, headline, source, link

    FinViz date/time format:
    - First row of a day: "Apr-30-25 02:40AM"
    - Subsequent rows same day: "03:10AM" (date omitted)
    We carry forward the last seen date.
    """
    if use_cache and os.path.exists(out_csv):
        print(f"Loaded FinViz headlines from cache: {out_csv}")
        return pd.read_csv(out_csv)

    url = f"https://finviz.com/quote.ashx?t={ticker}"
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                      "AppleWebKit/537.36 (KHTML, like Gecko) "
                      "Chrome/120.0 Safari/537.36"
    }

    r = requests.get(url, headers=headers, timeout=timeout)
    r.raise_for_status()

    soup = BeautifulSoup(r.text, "html.parser")
    news_table = soup.find("table", class_="fullview-news-outer")
    if news_table is None:
        raise RuntimeError(f"No FinViz news table found for {ticker}: {url}")

    rows = news_table.find_all("tr")

    os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)

    data = []
    current_date_str = None  # FinViz uses like "Apr-30-25"

    for row in rows:
        cols = row.find_all("td")
        if len(cols) < 2:
            continue

        dt_text = cols[0].get_text(" ", strip=True)
        # headline + link
        a = cols[1].find("a")
        headline = a.get_text(" ", strip=True) if a else cols[1].get_text(" ", strip=True)
        link = a["href"] if (a and a.has_attr("href")) else ""

        # source (usually in <span>)
```

FinViz headline scraping with date carry-forward for rows where only time is shown.

After combining the two sources, the news dataset was filtered again so that only items inside the stock price date range were kept. In the final run, this produced 73 combined news rows spanning 2025-01-15 to 2026-04-03

```
Combined news rows: 73

Combined news rows: 73
News date range: 2025-01-15 -> 2026-04-03
```

2.2 Sentiment analysis methods

Two sentiment approaches were used:

- VADER, a lightweight rule-based sentiment analyzer
- FinBERT, a transformer-based sentiment model trained for financial text

```python
def add_vader_sentiment(df_news: pd.DataFrame) -> pd.DataFrame:
    analyzer = SentimentIntensityAnalyzer()
    df = df_news.copy()
    df["vader_compound"] = df["headline"].astype(str).apply(lambda x: analyzer.polarity_scores(x)["compound"]
    return df

def aggregate_daily_sentiment(df_news_scored: pd.DataFrame) -> pd.DataFrame:
    daily = (
        df_news_scored.groupby("date")
        .agg(
            sent_mean=("vader_compound", "mean"),
            sent_sum=("vader_compound", "sum"),
            sent_count=("vader_compound", "count"),
        )
        .reset_index()
        .sort_values("date")
    )
    return daily
```

VADER was applied headline by headline and produced a vader_compound score. These scores were then aggregated by day into:

- sent_mean
- sent_sum
- sent_count

FinBERT was also applied at headline level. For each headline, the model output sentiment probabilities for negative, neutral, and positive classes. A continuous score was then created as:

finbert_score = P(positive) - P(negative)

These FinBERT scores were then aggregated daily into:

- finbert_mean
- finbert_sum
- finbert_count

This design allowed a direct comparison between a simple sentiment baseline and a more advanced finance-specific approach.

```python
def add_finbert_sentiment(
    df_news: pd.DataFrame,
    cache_csv: str = "news_cache/news_scored_finbert.csv",
    use_cache: bool = True,
    batch_size: int = 16,
    model_name: str = "ProsusAI/finbert"
) -> pd.DataFrame:
    """
    Run FinBERT on each headline and store per-headline sentiment probabilities.
    Creates finbert_score = P(pos) - P(neg) in [-1, 1].
    Caches output to CSV to avoid rerunning model inference.
    """
    os.makedirs(os.path.dirname(cache_csv) or ".", exist_ok=True)

    if use_cache and os.path.exists(cache_csv):
        print(f"Loaded FinBERT scored headlines from cache: {cache_csv}")
        return pd.read_csv(cache_csv)

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(model_name)
    model.eval()

    # label order in ProsusAI/finbert is usually: [negative, neutral, positive]
    id2label = model.config.id2label
    # Build mapping robustly
    label_to_index = {v.lower(): k for k, v in id2label.items()}

    headlines = df_news["headline"].astype(str).tolist()

    rows = []
    with torch.no_grad():
        for i in range(0, len(headlines), batch_size):
            batch = headlines[i:i + batch_size]
            enc = tokenizer(batch, padding=True, truncation=True, return_tensors="pt")
            out = model(**enc)
            probs = torch.softmax(out.logits, dim=1).cpu().numpy()

            for j, text in enumerate(batch):
                p_neg = float(probs[j][label_to_index.get("negative", 0)])
                p_neu = float(probs[j][label_to_index.get("neutral", 1)])
                p_pos = float(probs[j][label_to_index.get("positive", 2)])

                rows.append({
                    "date": df_news.iloc[i + j]["date"],
                    "headline": text,
                    "source": df_news.iloc[i + j].get("source", "")
```

FinBERT-based sentiment scoring used as the independent research enhancement.

2.3 Key functions and modules added for Task C.7

To improve maintainability, the original single script was separated into modules. The final structure included:

- data_sources.py for price/news downloading and cleaning
- sentiment.py for VADER and FinBERT scoring
- features.py for price features, labels, and sentiment merging
- evaluation.py for time split and evaluation metrics
- models.py for prediction helpers
- plots.py for chart generation
- main_c7.py as the pipeline runner

This structure made the code easier to debug and aligned better with the reporting requirement for well-documented code

3. Less Straightforward Design Decisions

3.1 Time alignment between news and stock data

One of the most important parts of this task was correctly aligning sentiment data with stock data. News articles do not appear every trading day, and the number of headlines per day is inconsistent. To solve this, sentiment was first computed at the headline level, then aggregated into one daily row per date. Those daily rows were merged onto the stock feature table using the trading date as the key. Missing sentiment values were filled with neutral defaults such as 0.0 for mean/sum and 0 for counts.

This method ensured that the model always had a full feature vector for each stock trading day, even on days when no relevant news was found.

```python
def merge_sentiment_features_both(
    df_feat: pd.DataFrame,
    df_daily_vader: pd.DataFrame,
    df_daily_finbert: pd.DataFrame
) -> pd.DataFrame:
    """
    Merge BOTH VADER and FinBERT daily sentiment features into the price feature table.
    Fill missing values with neutral defaults.
    """
    df = df_feat.merge(df_daily_vader, on="date", how="left")
    df = df.merge(df_daily_finbert, on="date", how="left")

    # VADER fill
    df["sent_mean"] = df["sent_mean"].fillna(0.0)
    df["sent_sum"] = df["sent_sum"].fillna(0.0)
    df["sent_count"] = df["sent_count"].fillna(0)

    # FinBERT fill
    df["finbert_mean"] = df["finbert_mean"].fillna(0.0)
    df["finbert_sum"] = df["finbert_sum"].fillna(0.0)
    df["finbert_count"] = df["finbert_count"].fillna(0)

    return df
```

Merging daily VADER and FinBERT features into the stock feature table.

3.2 Label construction for next-day direction

The classification label was created using the next day's close relative to the current day's close. Specifically:

- close_next = Close.shift(-1)
- label_up = 1 if close_next > Close else 0

This means the model predicts whether the next trading day will close higher than the current day. This matches the task requirement exactly because it converts the problem into a directional prediction rather than a price regression problem.

```python
def build_price_features(df_price: pd.DataFrame) -> pd.DataFrame:
    """
    Create basic price-derived features for classification.
    Uses only information available up to day t.
    """
    df = df_price.copy()
    df["date"] = df.index.strftime("%Y-%m-%d")

    # 1-day return
    df["ret_1d"] = df["Close"].pct_change()

    # moving averages
    df["ma_5"] = df["Close"].rolling(5).mean()
    df["ma_10"] = df["Close"].rolling(10).mean()

    # volatility (std of returns)
    df["vol_5"] = df["ret_1d"].rolling(5).std()

    # volume change
    df["volchg_1d"] = df["Volume"].pct_change()

    # label: UP/DOWN tomorrow
    df["close_next"] = df["Close"].shift(-1)
    df["label_up"] = (df["close_next"] > df["Close"]).astype(int)

    # Drop rows with NaNs caused by rolling/pct_change/shift
    df = df.dropna().reset_index(drop=True)

    return df
```

Construction of price-based input features and next-day UP/DOWN label.

3.3 Why both VADER and FinBERT were used

VADER was chosen as a baseline because it is easy to apply, fast to run, and commonly used in sentiment analysis experiments. However, VADER is not finance-specific and may not interpret financial language accurately.

FinBERT was added as the **independent research component** because it is specifically trained for financial sentiment. In theory, this should make it more suitable for market-related text. Comparing VADER and FinBERT made it possible to test whether a more sophisticated domain-specific model would improve performance in practice.

3.4 Time-based train/test split and evaluation metrics

Because stock data is time series data, a random train/test split would risk data leakage. Instead, the dataset was split chronologically using a simple time-based split, where the first 80% of rows were used for training and the final 20% for testing.

The evaluation used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

These metrics were chosen because directional prediction is a classification task and it is important to assess performance from multiple angles rather than relying only on accuracy.

```python
def time_split(df: pd.DataFrame, train_ratio: float = 0.8):
    n = len(df)
    split = int(n * train_ratio)
    train_df = df.iloc[:split].copy()
    test_df = df.iloc[split:].copy()
    return train_df, test_df

def eval_classifier(y_true, y_pred, title="Model"):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    cm = confusion_matrix(y_true, y_pred)

    print(f"\n--- {title} ---")
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1:        {f1:.4f}")
    print("Confusion matrix:\n", cm)

    return {"acc": acc, "prec": prec, "rec": rec, "f1": f1, "cm": cm}
```

Time-based train/test split and classification evaluation metrics.

4. References Used

The following tools, datasets, and libraries were used in this task:

- Yahoo Finance via yfinance for historical price data
- FinViz for financial headlines
- Alpha Vantage News Sentiment API for additional news coverage
- vaderSentiment for rule-based sentiment analysis
- ProsusAI/finbert from Hugging Face Transformers for finance-specific sentiment analysis
- scikit-learn for Logistic Regression, Random Forest, and evaluation metrics
- matplotlib for visualisation support

5 Experiment Results Summary

5.1 Dataset summary

The final run produced:

- 317 stock price days
- 73 combined news rows after filtering
- 34 daily VADER sentiment rows
- 33 daily FinBERT sentiment rows
- 24 overlapping dates between price days and sentiment days

A final classification dataset of 307 rows and 19 columns

The label distribution was reasonably balanced:

- 55.37% UP
- 44.63% DOWN

This balance is useful because the classification problem is not extremely skewed toward one class.

5.2 Model comparison results

Two classifiers were tested:

- Logistic Regression
- Random Forest

```python
# X_train_sent = train_df[price_cols + sent_cols].values
# X_test_sent = test_df[price_cols + sent_cols].values

y_train = train_df["label_up"].values
y_test = test_df["label_up"].values

# Baseline model: Logistic Regression
lr_base = LogisticRegression(max_iter=2000)
lr_base.fit(X_train_base, y_train)
pred_base = lr_base.predict(X_test_base)
m_base = eval_classifier(y_test, pred_base, title="LogReg (Price-only)")

# --- Logistic Regression comparisons ---
lr_vader = LogisticRegression(max_iter=2000)
lr_vader.fit(X_train_vader, y_train)
pred_lr_vader = lr_vader.predict(X_test_vader)
m_lr_vader = eval_classifier(y_test, pred_lr_vader, title="LogReg (Price + VADER)")

lr_finbert = LogisticRegression(max_iter=2000)
lr_finbert.fit(X_train_finbert, y_train)
pred_lr_finbert = lr_finbert.predict(X_test_finbert)
m_lr_finbert = eval_classifier(y_test, pred_lr_finbert, title="LogReg (Price + FinBERT)")

lr_both = LogisticRegression(max_iter=2000)
lr_both.fit(X_train_both, y_train)
pred_lr_both = lr_both.predict(X_test_both)
m_lr_both = eval_classifier(y_test, pred_lr_both, title="LogReg (Price + VADER + FinBERT)")
```

Experimental setup for comparing price-only and sentiment-enhanced classifiers.

For Logistic Regression, the results were:

- Price-only: Accuracy = 0.5484, F1 = 0.6216
- Price + VADER: Accuracy = 0.5484, F1 = 0.6216
- Price + FinBERT: Accuracy = 0.5323, F1 = 0.6027
- Price + VADER + FinBERT: Accuracy = 0.5323, F1 = 0.6027

```
--- LogReg (Price-only) ---
Accuracy:  0.5484
Precision: 0.5897
Recall:    0.6571
F1:        0.6216
Confusion matrix:
 [[11 16]
 [12 23]]
```

```
--- LogReg (Price + VADER) ---
Accuracy:  0.5484
Precision: 0.5897
Recall:    0.6571
F1:        0.6216
Confusion matrix:
 [[11 16]
 [12 23]]
```

```
--- LogReg (Price + FinBERT) ---
Accuracy:  0.5323
Precision: 0.5789
Recall:    0.6286
F1:        0.6027
Confusion matrix:
 [[11 16]
 [13 22]]

--- LogReg (Price + VADER + FinBERT) ---
Accuracy:  0.5323
Precision: 0.5789
Recall:    0.6286
F1:        0.6027
Confusion matrix:
 [[11 16]
 [13 22]]
```

For Random Forest, the results were:

- Price-only: Accuracy = 0.5323, F1 = 0.5672
- Price + VADER: Accuracy = 0.5645, F1 = 0.6197
- Price + FinBERT: Accuracy = 0.5323, F1 = 0.5672

```
--- RandomForest (Price-only) ---
Accuracy:  0.5323
Precision: 0.5938
Recall:    0.5429
F1:        0.5672
Confusion matrix:
 [[14 13]
 [16 19]]

--- RandomForest (Price + VADER) ---
Accuracy:  0.5645
Precision: 0.6111
Recall:    0.6286
F1:        0.6197
Confusion matrix:
 [[13 14]
 [13 22]]
```

These results show two important findings:

1. Logistic Regression (Price-only) gave the highest overall F1 score at 0.6216
2. Random Forest improved noticeably when VADER sentiment was added, increasing F1 from 0.5672 to 0.6197

5.3 Interpretation of results

The results suggest that sentiment features can add value, but the effect depends on the model and the sentiment method used.

For Logistic Regression, adding sentiment did not improve results. In fact, the VADER version matched the price-only baseline exactly, while FinBERT slightly reduced performance. For Random Forest, VADER sentiment provided a clear improvement over the price-only model, while FinBERT again did not improve performance.

This suggests that:

- sentiment can be useful in some settings
- A more advanced sentiment model does not automatically lead to better classification results
- feature usefulness depends on both the model type and the amount/quality of available sentiment data

The final demonstration prediction using RandomForest (Price + VADER) predicted DOWN with a probability of UP equal to 0.4285, but the actual next-day label was UP. This single incorrect prediction does not invalidate the model. The overall evaluation is more important than one example, especially since the model's F1 score is only moderate rather than close to perfect.

```
Final demo model: RandomForest (Price + VADER)

Next trading day direction prediction:
Direction: DOWN
Predicted label (1=UP, 0=DOWN): 0
Probability of UP: 0.4285
Last test day (date): 2026-04-01
Close on last test day: 171.89999389648438
Actual next-day label (1=UP,0=DOWN): 1
```

5.4 Challenges and limitations

Several practical issues were encountered during this task.

First, the modular refactor initially failed due to import and consistency problems, including a missing c7_utils module and a missing pandas import in evaluation.py. These issues were later corrected so that the full pipeline could run successfully.

Second, sentiment coverage was relatively sparse compared with the number of trading days. Although there were 317 price days, there were only 34 VADER sentiment days and 24 overlapping days between price and sentiment. This means that many trading days used neutral-filled sentiment values, which likely reduced the impact of sentiment features.

Third, not all sentiment sources were equally informative. In this experiment, FinBERT did not outperform VADER, even though it is a more advanced finance-specific model. One possible explanation is that the dataset was still too small or too noisy for the extra model complexity to provide an advantage.

6. Visualisations Included

Two visualisations were generated to support the interpretation of the sentiment-based classification results.

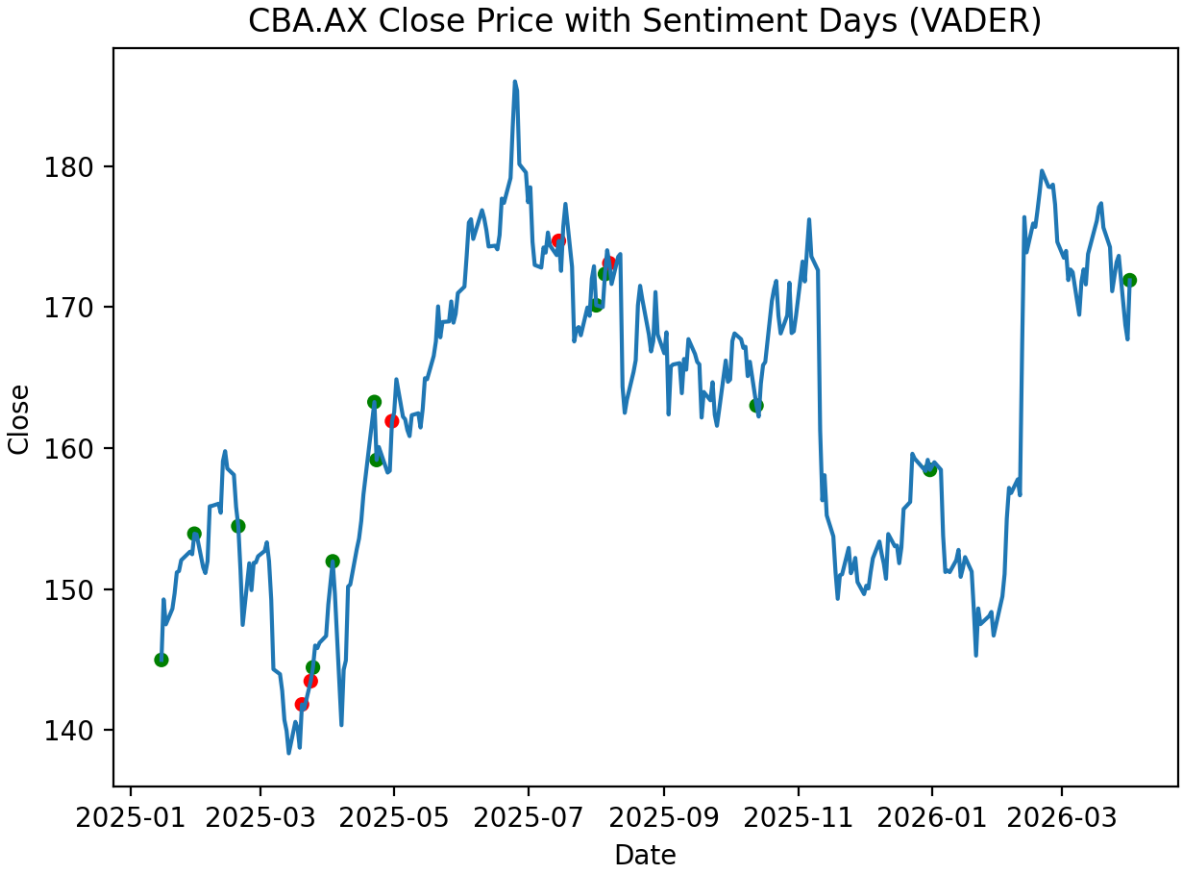

Figure 1. CBA.AX Close Price with Sentiment Days (VADER)

This figure shows the daily closing price of CBA.AX over the experiment period, with markers highlighting days where relevant news sentiment was available. Green markers indicate positive daily VADER sentiment, while red markers indicate negative daily VADER sentiment.

The plot helps illustrate two important observations. First, sentiment data was relatively sparse compared with the full trading timeline, as only selected days contained matched news headlines after filtering and aggregation. Second, sentiment events were distributed unevenly across the time period rather than occurring consistently on every trading day. This helps explain why sentiment features did not improve every model equally: most trading days had no strong news signal and therefore relied mainly on price-based inputs.

Overall, this figure supports the interpretation that sentiment was available only on a limited subset of trading days, which likely reduced the overall contribution of sentiment features in the final classification models.

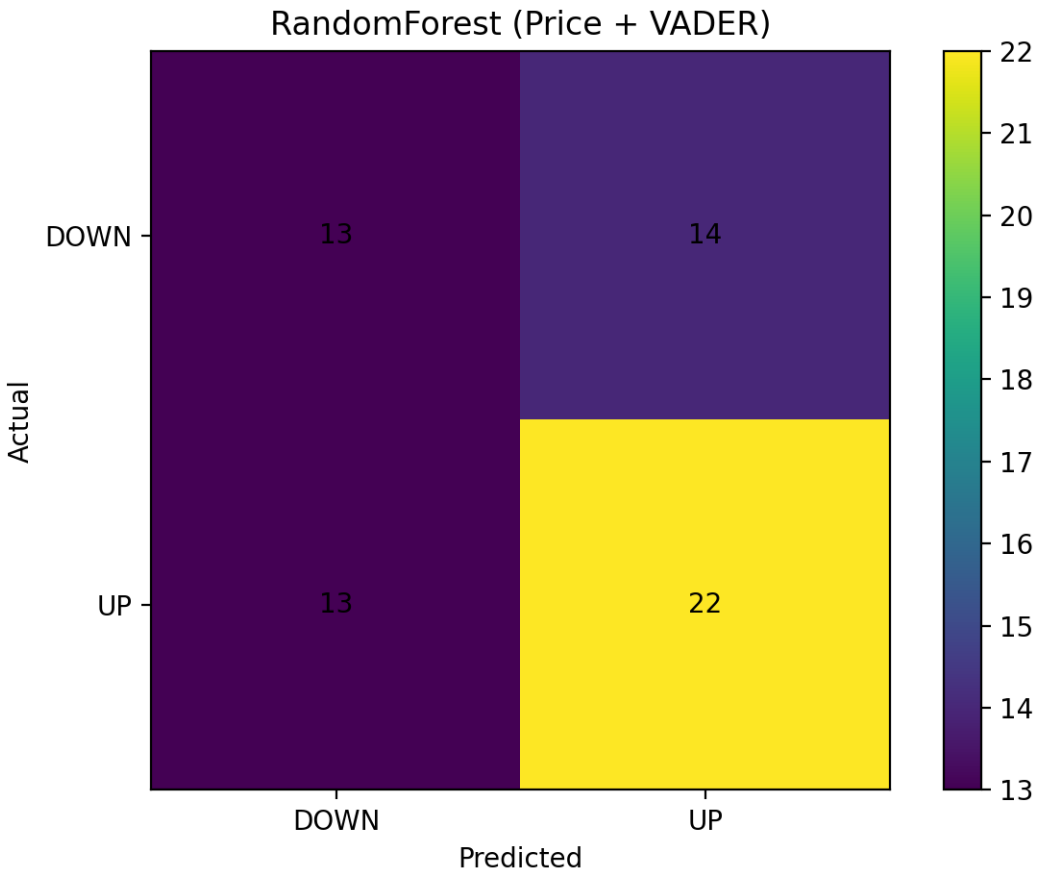

Figure 2. Confusion Matrix for RandomForest (Price + VADER)

This figure presents the confusion matrix for the RandomForest (Price + VADER) classifier, which was the best-performing Random Forest configuration in the experiment.

The matrix shows that:

- 13 actual DOWN days were correctly predicted as DOWN
- 14 actual DOWN days were incorrectly predicted as UP
- 13 actual UP days were incorrectly predicted as DOWN
- 22 actual UP days were correctly predicted as UP

This result indicates that the model performed better at identifying UP days than DOWN days, as shown by the higher number of correct UP predictions. However, the classifier still made a noticeable number of mistakes in both directions, which is consistent with the moderate overall F1-score rather than near-perfect predictive performance.

The confusion matrix is useful because it shows more detail than accuracy alone. It reveals that the model has some predictive value but still struggles with class separation, especially when distinguishing borderline DOWN cases from UP cases.

7. Conclusion

This task successfully extended the stock prediction system from price forecasting to sentiment-enhanced next-day direction classification. Historical price features were combined with external news sentiment from FinViz and Alpha Vantage, and two sentiment analysis methods, VADER and FinBERT, were evaluated. The final experiments showed that the best overall result came from Logistic Regression (Price-only) with F1 = 0.6216, while Random Forest (Price + VADER) showed the clearest evidence that sentiment could improve performance, increasing F1 from 0.5672 to 0.6197.

Overall, the results indicate that sentiment can provide useful additional information, but its value depends on how it is represented and which classification model is used. The independent research component using FinBERT was technically successful, even though it did not outperform VADER on this dataset. This outcome is still meaningful because it shows that more sophisticated methods are not always better in practice.